CargoBot26: SPRINT0

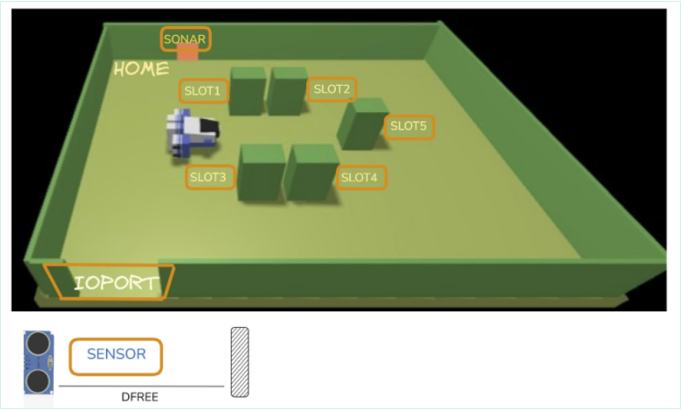
Introduction

GOAL:
- Chiarire i requisiti del committente e formalizzarli.
- Si stabilisce il piano di lavoro con scadenze e Sprint successivi.

Requirements

A *Maritime Cargo* shipping company (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**).

The *hold* is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides 4 **slots** to store the containers and a slot named **slot5**.



In the picture above:

• The **slots1-4** depict the hold areas reserved to store one *container* each,

• The **slot5** depicts an area, where the *cargorobot* must temporarily store a container, before to place it in one of the *slots1-4*. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed.

• The **IOPort** is a device with a **pushbutton** and a **display**.
The *pushbutton* is pressed by the customer in order to send a request to load a container on the cargo. The *display* is used to show the answer to the request and to show the current state of the *hold*.

• The **sensor** associated to the *IOPort* is a device (a *sonar*) used to detect the presence of a container, when it measures a distance D, such that D < DFREE/2, during a reasonable time (e.g. 3 secs).

The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

• It sends the answer retrylater, if the **IOPort** is currently occupied by a container or if the system is *Out of service*

• It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.

• Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot.
While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the **slot5** (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

• the current state of the *hold*

• the message **'Service working'**, when it is all is going well

• the message **'Out of service'** if the *sonar sensor* measures a distance D > DFREE for at least 3 secs
(perhaps a failure of the *sonar*).

Requirement Analysis

Una società di trasporto che chiameremo semplicemente ***company*** intende automatizzare le operazioni di carico dei container tramite l'utilizzo di un DDR.

• ***cargorobot*** rappresenta l'entità robotica che applica al DDR i servizi richiesti dai requisiti del committente.

Il ***DDR*** è fornito dal committente tramite software 'VirtualRobot26' che simula un *Differential Drive Robot* in un ambiente virtuale denominato WEnv.
Questo modello è progettato per essere facilmente adattabile ad un robot reale.
Documentazione relativa al Robot Virtuale

Il committente rende dispoibili ulteriori componenti software dedicati al DDR.

In particolare, 'RobortSmart26' riesce ad adattarsi al contesto del problema in quanto sviluppato come un servizio capace di comunicare con altre entità.
Si adegua alle richieste del committente poichè capace di rappresentare la stiva tramite una mappa mentale e di pianificare
come raggiungere una posizione obbiettivo tramite opportuni algoritmi.

Dalla documentazione emerge che il software RobotSmart26 dispone già di una propria rappresentazione interna di mappa, che nel caso in esame
corrisponde alla stiva della compagnia marittima studiata. Non è pertanto necessario effettuare ulteriori operazioni di ridimensionamento,
ferma restando la possibilità di modificarne la geometria qualora se ne presenti la necessità.
Le coordinate della mappa e quelle utilizzate per il movimento del robot sono espresse in unità robotiche su un piano bidimensionale (X, Y).
Nel seguito, si farà riferimento a tale sistema di coordinate e al relativo dimensionamento.

Nel complesso ***cargorobot*** è un'entità proattiva capace di ricevere/inviare messaggi ed eseguire movimenti sul DDR.
In una futura analisi, sarà necessario capire se le funzionalità offerte da 'RobortSmart26' sono sufficienti a soddisfare le richieste del committente.

- La ***hold*** (stiva) viene modellata come un'area rettangolare che costituisce il riferimento spaziale per descrivere la posizione di alcuni elementi
del sistema all'interno della rappresentazione interna della mappa utilizzata da RobotSmart26. Il concetto di hold risulta quindi fondamentale per formalizzare la collocazione di tali elementi
rispetto alla mappa del software, garantendo una corrispondenza tra le posizioni logiche del sistema e le coordinate robotiche.

- ***HOME*** -- posizione di stazionamento del DDR
- ***IOPort*** -- zona di carico dei container (così chiamata nei requisiti)
- ***slot*** -- come area di parcheggio

Dai requisiti si evince che è necessario formalizzare un'associazione tra il nome identificativo di uno slot (slotId) e la sua posizione.
Di seguito viene mostrata la corrispondenza tra la stringa che identifica lo slot e la sua posizione in coordinate robotiche:

Stringa Identificativa (Slot)	**Posizione / Coordinate Robotiche (X, Y)**
slot1	(1, 1)
slot2	(1, 4)
slot3	(3, 1)
slot4	(3, 4)
slot5	(2, 5)

Viene formalizzata anche la posizione dell'IOPort e della HOME:

Componente	**Posizione / Coordinate Robotiche (X, Y)**
IOPort	(4, 0)
HOME	(0, 0)

Per rappresentare in modo strutturato le coordinate utilizzate da RobotSmart26, si introduce la classe POJO **Position**,
che incapsula le coordinate intere X e Y in un unico oggetto.
Questa scelta consente di trattare una posizione come un'entità distinta, anziché gestire le due coordinate separatamente.

```
  public class Position {
    private final int x;
    private final int y;

    public Position(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // restituisce il valore della coordinata X come intero
    public int getX() {
        return x;
    }

    // restituisce il valore della coordinata Y come intero
    public int getY() {
        return y;
    }

    @Override
    public boolean equals(Object o) {
        Position position = (Position) o;
        return x == position.x && y == position.y;
    }

    @Override
    // restituisce il valore delle coordinate come stringa (X,Y)
    public String toString() {
        return "(" + x + "," + y + ')';
    }
}
```

Attualmente la ***hold*** non presenta alcun comportamento proattivo; dunque, potrà essere rappresentata tramite un POJO.

```
public interface IHold {

  // -- HOME:
    // restituisce la posizione della HOME in coordinate RobotSmart (X,Y)
    Position getHomePosition();

  //--IO-PORT:
    // restituisce la posizione dell'IOPort in coordinate RobotSmart (X,Y)
    Position getIOPortPosition();

  //--SLOT:
    // restituisce lo slot identificato dal nome
    ISlot getSlot(String slotId);

    // verifica se esiste uno slot libero
    boolean hasFreeSlot();

    // restituisce la posizione dello slot in coordinate RobotSmart (X,Y)
    Position getSlotPosition(String slotId);

    // restituisce il primo slot libero.
    ISlot getFirstFreeSlot();

}
```

• Uno ***slot*** rappresenta una area di parcheggio di container all'interno della ***hold***.

Dai requisiti emerge che ogni slot può assumere tre distinti stati: occupato, libero e riservato.

Gli ***slot*** sono in totale cinque. Lo "***slot5***" è riservato al processo di etichettatura di un container.
Anche questa entità non ha alcun comportamento proattivo, ragion per cui verrà modellata come un POJO. L'area rappresentata da uno ***slot*** non è mobile;
le sue coordinate sono strettamente dipendenti dalla ***hold***.

```
public enum SlotStatus {
  FREE,
  OCCUPIED,
  RESERVED
}
```

```
public interface ISlot {

    // restituisce l'id dello slot
    String getId();

    // restituisce lo stato dello slot
    SlotStatus getSlotStatus();

    // imposta lo stato dello slot
    void setSlotStatus(SlotStatus status);
}
```

• Da requisiti il concetto di ***IOPort*** presenta due significati differenti.

1. 'IOPort' intesa come area di carico dei container.
2. 'IOPort' come una Web-GUI che comunica attraverso un protocollo a messaggi asincrono.

Per evitare ambiguità, d'ora in avanti il secondo significato sarà indicato con il termine Web-GUI o con espressioni equivalenti,
mentre IOPort farà riferimento all'area fisica destinata alle operazioni sui container, salvo diversa specificazione.

La Web-GUI presenta due elementi funzionali:

- ***pushbutton*** viene premuto dal cliente per inviare una richiesta di carico; non ha alcun comportamento proattivo.
- ***display*** viene utilizzato come dispositivo di output di visualizzazione di messaggi dal ***cargoservice***; non ha alcun comportamento proattivo.

• ***sonar*** è un componente hardware proattivo del sistema che ha il compito di misurare in maniera continua una determinata distanza e generare eventi.

Il committente vuole che questo sia integrato tramite un PicoW.

- ***D*** distanza rilevata dal sonar
- ***DFREE*** riferimento di distanza rilevata

Le sue caratteristiche portano a modellare il sonar come un componente proattivo in grado di interagire con il sistema comunicando informazioni.

• ***LED*** è un componente hardware in grado di emettere luce.
In accordo con il committente, il LED farà parte dell'architettura del PicoW.

Macro-componenti individuati:

• ***cargoservice***
- Si occupa della logica di business relativa alle richieste di movimentazione e allocazione dei container.
- Presenta un comportamento proattivo in grado di aggiornare i dispositivi di output come display e led.
- È capace di comunicare e percepire messaggi ed eventi da altre entità.

• ***cargorobot***
- Il componente che realizza le operazioni richieste relative al DDR.
- Presenta un comportamento proattivo, è capace di navigare autonomamente all'interno della hold.
- Per l'esecuzione dei movimenti, utilizza come servizio esterno il software RobotSmart26.
- È capace di comunicare e percepire messaggi ed eventi da altre entità.

• ***webgui***
- Il componente che si interfaccia con l'utente.
- In input un bottone, in output un display.
- È capace di comunicare e percepire messaggi ed eventi da altre entità.

• ***sonar***
- Rappresenta il componente hardware sonar del PicoW.

• ***led***
- Rappresenta il componente hardware led PicoW.

Modello e Nodi Computazionali:

Il sistema viene concepito come un sistema distribuito, in cui i processi e le entità software sono ripartiti tra più nodi computazionali.
In particolare, si individuano almeno due nodi distinti: uno dedicato all'esecuzione dei servizi software
e uno dedicato alla gestione dell'hardware basato su Raspberry Pi Pico W (PicoW).

Questa separazione è motivata dai limiti delle risorse computazionali del PicoW, che non risulterebbero sufficienti a sostenere autonomamente
l'esecuzione dell'intero insieme di processi e la gestione di tutte le entità software previste.
È pertanto necessario affiancargli un'ulteriore macchina, dotata di maggiori risorse computazionali, sulla quale eseguire i servizi applicativi.

Individuiamo i seguenti **nodi**:

• Un nodo dedicato all'esecuzione dei sevizi software (VirtualRobot26, RobortSmart26, cargoservice, cargorobot, webgui).

• Un nodo fisico dedicato capace di gestire i componenti hardware quali sonar e led (PicoW).

Utilizzo di Qak:

Scegliamo di descrivere il sistema tramite modello **Qak** (prodotto tramite linguaggio/metamodello Qak) per semplificare il ciclo di vita del sistema da sviluppare.
Questo DSL ci consente di formalizzare il concetto di messaggio e di attore come servizio come segue: **Documentazione relativa a Qak**.
Inoltre, ci permette di colmare l'abstraction gap tra la visione del dominio e i componenti software concreti, garantendo uno sviluppo rapido e coerente.

Test plans

• Slot:

```
private ISlot slot;

@Before
public void setUp() {
  slot = null;
}

@Test
public void testGetId() {
  assertEquals("slot1", slot.getId());
}

@Test
public void testDefaultIsFree() {
      // uno slot appena creato è libero di default
      assertEquals(SlotStatus.FREE, slot.getSlotStatus());
}

@Test
public void testSetIsOccupied() {
      slot.setSlotStatus(SlotStatus.OCCUPIED);
      assertEquals(SlotStatus.OCCUPIED, slot.getSlotStatus());
}

@Test
public void testSetIsFree() {
      slot.setSlotStatus(SlotStatus.FREE);
      assertEquals(SlotStatus.FREE, slot.getSlotStatus());
}
```

• Hold:

```
private IHold hold;

@Before
  public void setUp() {
      hold = new Hold();
}

@Test
  public void testGetSlot() {
      // slot inizializzati da 1 a 5
      ISlot s = hold.getSlot("slot1");
      assertNotNull(s);
      assertEquals("slot1", s.getId());
}

@Test
  public void testHasFreeSlotWhenAllFree() {
      // slot inizializzati sono liberi
      assertTrue(hold.hasFreeSlot());
}

@Test
  public void testHasFreeSlotWhenAllOccupied() {
      // i requisiti indicano che gli slot da riservare sono slot1-4 (slot5 è per il marker)
      for (int i = 1; i <= 4; i++) {
          hold.getSlot("slot" + i).setSlotStatus(SlotStatus.OCCUPIED);
      }
      assertFalse(hold.hasFreeSlot());
}

@Test
  public void testSlotPositions() {
      // le posizioni degli slot sono inizializzate direttamente dalla Hold
      assertEquals(new Position(1, 1), hold.getSlotPosition("slot1"));
      assertEquals(new Position(1, 4), hold.getSlotPosition("slot2"));
      assertEquals(new Position(3, 1), hold.getSlotPosition("slot3"));
      assertEquals(new Position(3, 4), hold.getSlotPosition("slot4"));
      assertEquals(new Position(2, 5), hold.getSlotPosition("slot5"));
}

@Test
  public void testGetSlotPositionUnknownSlot() {
      assertNull(hold.getSlotPosition("slot99"));
}

@Test
  public void testGetFirstFreeSlot() {
      // assumiamo che la hold ritorni il primo slot libero
      hold.getSlot("slot1").setSlotStatus(SlotStatus.OCCUPIED);
      ISlot free = hold.getFirstFreeSlot();
      assertNotNull(free);
      assertEquals("slot2", free.getId());
}

@Test
  public void testGetFirstFreeSlotWhenFull() {
      for (int i = 1; i <= 4; i++) {
          hold.getSlot("slot" + i).setSlotStatus(SlotStatus.OCCUPIED);
      }
      assertNull(hold.getFirstFreeSlot());
}
```

Piano di lavoro

- ***SPRINT1***
  - Progettare e implementare la logica principale del sistema, ovvero la ricezione di una richiesta di carico e la sua esecuzione tramite il robot.
  - [Rif: cargoservice, cargorobot]
  - [Tempo stimato: 40 ore-uomo]
- ***SPRINT2***
  - Realizzazione e implementazione della logica relativa ai componenti fisici che dovranno interagire con il sistema
  - [Rif: PicoW]
  - [Tempo stimato: 35 ore-uomo]
- ***SPRINT3***
  - Sviluppare la Web-GUI e verificare il corretto funzionamento con le interazioni utente.
  - [Rif: webgui]
  - [Tempo stimato: 30 ore-uomo]

Team

Alessandro Quaranta - alessandro.quaranta@studio.unibo.it
Gaia Verzelli - gaia.verzelli@studio.unibo.it

Project GitHub repository:
https://github.com/bbnogaia/cargo-robot-2026

Student GitHub repository:
https://github.com/alequaranta/iss2026quaranta

